SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

K. J. Somaiya School of Engineering, Mumbai-77 (Somaiya Vidyavihar University)
Department of Computer Engineering · iOS App Development with SwiftUI · Semester VII · 2026-27

Somaiya TRUST

OpenParliament

A SwiftUI civic-transparency app that links members of Parliament, the bills they shape and the budgets those bills fund — served from a local PostgreSQL database.

Soumil Sinha
Mini Project
SwiftUI · Node.js · PostgreSQL

PROBLEM & SOLUTION

Attendance registers, bill texts, budget documents and audit reports are public, but scattered across PDFs and portals. **OpenParliament** gathers them into one calm, neutral app where every record links to the related ones: a bill shows who spoke on it and which scheme funds it; a scheme shows how much of its allocation was actually spent and what the auditor found.

Data lives in a **normalised PostgreSQL database**, reaches the app through a **REST API**, and is held in a single **@Observable store**. Following a bill writes straight to the database; if the server is down, the app falls back to bundled data.

AT A GLANCE

4 tabs · 9 screens

17 database tables

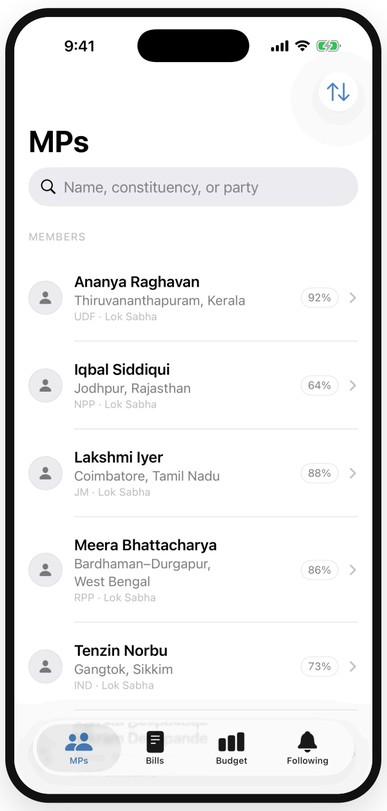
3 SQL views · 3 enums

13 REST endpoints

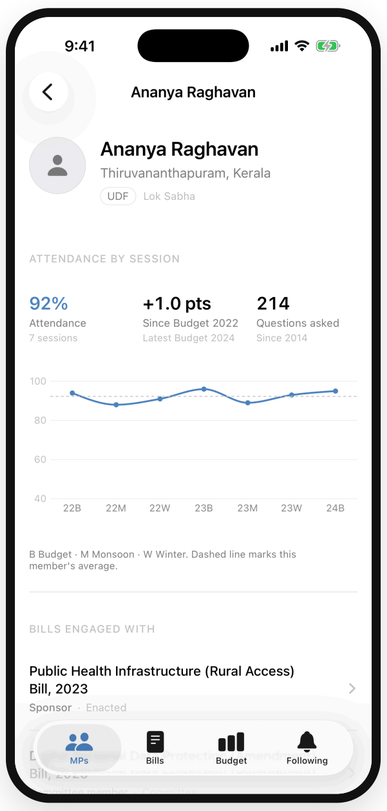
1 request loads everything

6/6 API tests passing

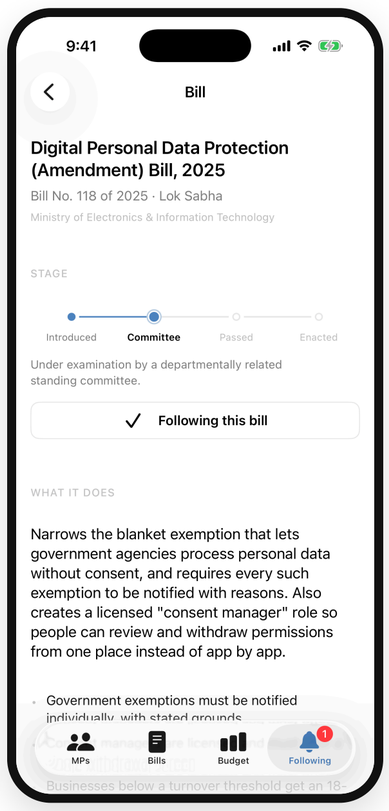
SCREENS



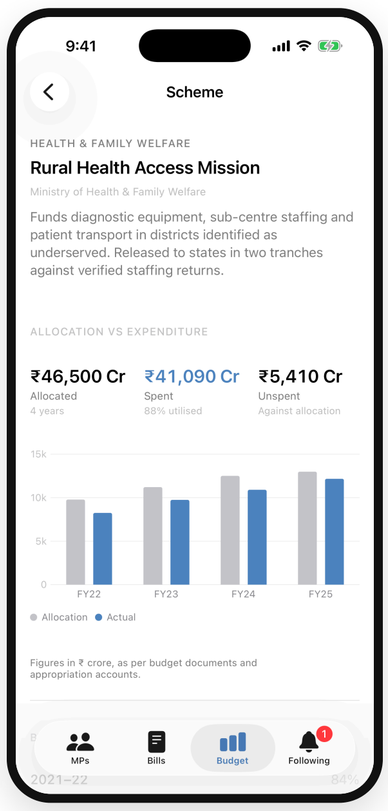
MPs
Search, sort, attendance badge



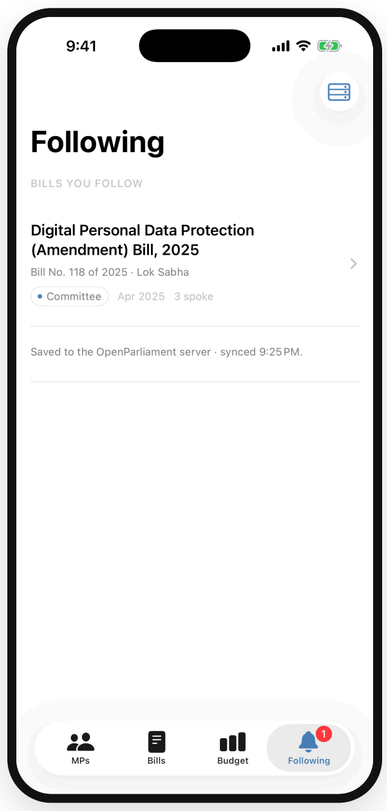
MP profile
Attendance chart, bills, speeches



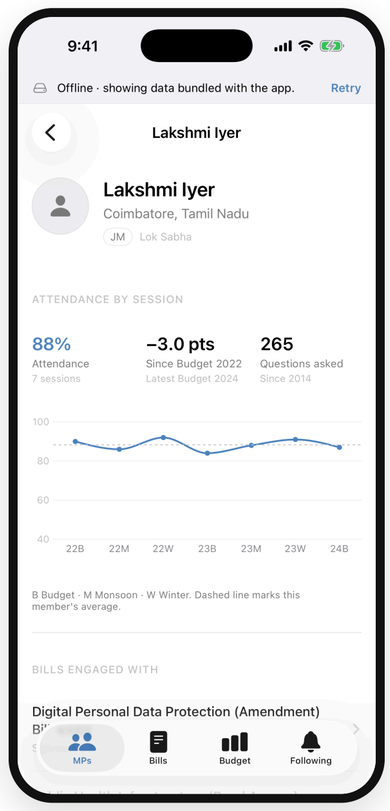
Bill detail
Stage stepper, follow button



Budget scheme
Allocation vs. actual spend



Following
Synced with PostgreSQL



Offline mode
Bundled data + retry banner

FUNCTIONS

Track members
Attendance across seven sessions, trend, questions asked, committees and every speech on record.

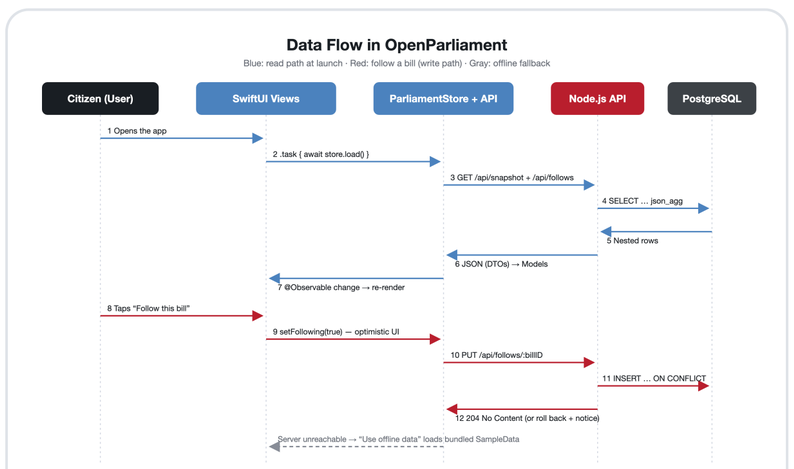
Follow legislation
Four-stage progress, plain-language summary, dated history, speakers and the schemes that fund it.

Audit the budget
Four years of allocation vs. expenditure, utilisation, unspent funds and CAG observations.

Follow & sync
Optimistic follow / unfollow written to **bill_follows**; rolled back with a notice if the write fails.

Resilient
Loading, error and offline states; server address configurable for running on a real iPhone.

DATA FLOW

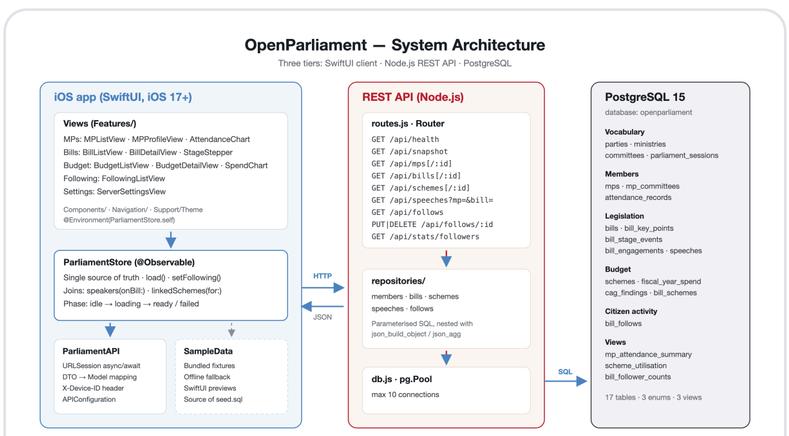


Read: launch → `GET /api/snapshot` → nested `json_agg` rows → models → views re-render.

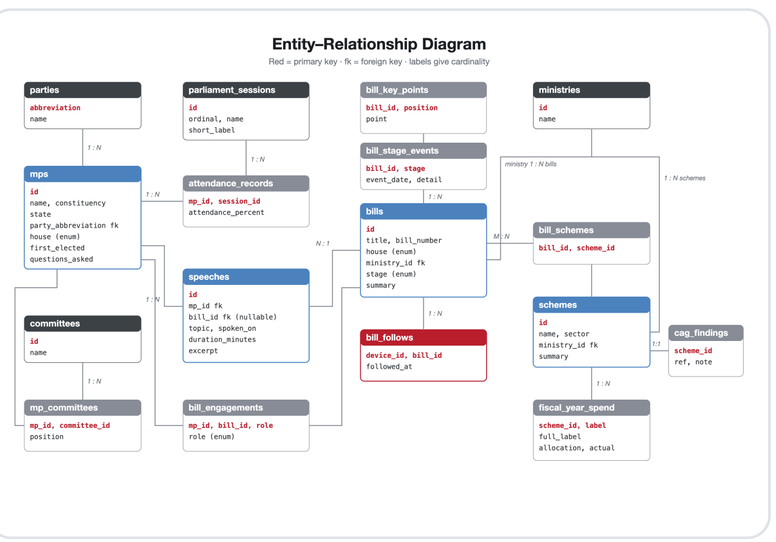
Write: tap Follow → UI updates at once → `PUT /api/follows/:id` → `INSERT … ON CONFLICT`.

Offline: server unreachable → "Use offline data" loads the bundled dataset.

ARCHITECTURE



DATABASE



TECHNOLOGY

SwiftUI · Observation · Swift Charts · NavigationStack · async / await · URLSession

Node.js · node-postgres · REST · JSON · PostgreSQL 15 · 3NF schema · Views & enums · node --test

CONCLUSION & FUTURE SCOPE

A static prototype became a complete three-tier product: relational storage, a thin API and a reactive SwiftUI client with offline resilience.

- Push alerts when a followed bill changes stage
- Import real open-data records · full-text search over speeches